

Paradigmi di Programmazione - A.A. 2022-23

Esame Scritto del 23/05/2023

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$(\lambda g.\lambda y.gyx)(\lambda f.fy)(\lambda z.z)$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

SOLUZIONE:

Una possibile soluzione:

$$\begin{aligned} & (\lambda g.\lambda y.gyx)(\lambda f.fy)(\lambda z.z) \\ \equiv_{\alpha} & (\lambda g.\lambda k.gkx)(\lambda f.fy)(\lambda z.z) \\ \rightarrow & (\lambda k.(\lambda f.fy)kx)(\lambda z.z) \\ \rightarrow & (\lambda f.fy)(\lambda z.z)x \\ \rightarrow & ((\lambda z.z)y)x \\ \rightarrow & yx \end{aligned}$$

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f x y z =  
  match (x,y) with  
  | (0,[]) -> []  
  | (x1,[]) -> z  
  | (x1,x2::lis) -> if x2<0 then lis else [];;
```

SOLUZIONE:

Struttura del tipo:

```
X -> Y -> Z -> RIS
```

Uso per convenzione X,Y,Z come variabili di tipo per i parametri x,y,z, RIS come variabile di tipo del risultato, e A,B,C,... come variabili di tipo "fresche" per la definizione dei vincoli.

Vincoli:

```
X = int           (da pattern matching)
Y = A list        (da pattern matching)
Z = RIS           (da secondo caso pattern matching)
A = int           (da tipo di x2 nella guardia dell'if)
Z = A list        (da terzo caso del pattern matching)
RIS = int list     (da terzo caso del pattern matching)
```

Ne consegue:

```
X = int
Y = int list
Z = int list
RIS = int list
```

Tipo inferito:

```
int -> int list -> int list -> int list
```

Esercizio 3 [Punti 7]

Assumendo il seguente tipo di dato che descrive alberi n-ari di interi:

```
type ntree =
  | Node of int * ntree list
```

Definire in OCaml le funzioni `sum` e `flat` con i seguenti tipi

```
sum : ntree -> int
flat : ntree -> int list
```

tali che:

- `sum` prende un albero e restituisce la somma di tutti i valori memorizzati nei suoi nodi
- `flat` esegue una **visita posticipata** dell'albero, ossia restituisce la lista di tutti i valori contenuti nei nodi tenendo conto che, per ogni nodo, la lista prodotta come risultato contiene prima i valori dei figli e poi il valore del nodo stesso.

Ad esempio, considerando il seguente albero:

```
let t = Node (1, [
  Node (2, []);
  Node (3, [
    Node (4, []);
    Node (5, [])
  ])
]);
```

```
Node (6, [])  
])
```

le funzioni danno i seguenti risultati

```
sum t;;      (* restituisce 21 *)  
flat t;;     (* restituisce [2;4;5;3;6;1] *)
```

SOLUZIONE:

Una possibile soluzione:

```
let rec sum t =  
  let aux s t' = s+(sum t') in  
  match t with  
  | Node(n,lis) -> List.fold_left aux n lis;;  
  
let rec flat t =  
  match t with  
  | Node(n,[]) -> [n]  
  | Node(n,t'::lis) -> (flat t')@(flat(Node (n,lis)));;
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con il costrutto **Seq** per la definizione di una struttura dati per memorizzare sequenze di valori non modificabili. Una sequenza viene creata definendone il contenuto e può essere processata solo tramite le seguenti operazioni **merge** e **map**:

- **merge(seq1,seq2)**: date due espressioni **seq1** e **seq2** che descrivono sequenze, restituisce una nuova sequenza ottenuta concatenando le due. Ad esempio (con una sintassi simile a OCaml):

```
merge ( Seq [1;2;3] , Seq [4;5;6] ) restituisce il valore [1; 2; 3; 4; 5; 6].
```

- **map(seq,x,body)**: data una espressione che descrive un sequenza **seq**, un identificatore **x** e una espressione **body** che utilizza **x**, restituisce una nuova sequenza ottenuta valutando ripetutamente **body** sostituendo ad **x** ogni elemento di **seq**, uno alla volta nell'ordine in cui sono in **seq**. Ad esempio (con una sintassi simile a OCaml):

```
map ( Seq [1;2;3] , x , x+1 ) restituisce il valore [2; 3; 4].
```

Si mostri come deve essere modificato l'interprete OCaml del linguaggio.

SOLUZIONE:

Una possibile soluzione:

```
type exp = ...
  | Seq of exp list
  | Merge of exp * exp
  | Map of exp * ide * exp

type evT = ...
  | SeqVal of evT list

let rec eval e s = match e with
  ...
  | Seq(lis) -> let eval2 s e = eval e s in
    SeqVal(List.map (eval2 s) lis)
  | Merge(e1,e2) -> let seq1 = eval e1 s in
    let seq2 = eval e2 s in
    ( match (seq1,seq2) with
      | (SeqVal(lis1) , SeqVal(lis2)) -> SeqVal(lis1@lis2)
      | _ -> failwith "Error"
    )
  | Map(e,x,body) -> let seq = eval e s in
    (match seq with
    | SeqVal(lis) ->
      let step y = let s' = bind s x y in
        eval body s'
      in SeqVal(List.map step lis)
    | _ -> failwith "Error"
    )
  )
```